MEDTRACK: AWS Cloud-Enabled Healthcare Management System

Project Description:

In today's fast-evolving healthcare landscape, efficient communication and coordination between doctors and patients are crucial. Med Track is a cloud-based healthcare management system that streamlines patient doctor interactions by providing a centralized platform for booking appointments, managing medical histories, and enabling diagnosis submissions. To address these challenges, the project utilizes Flask for backend development, AWS EC2 for hosting, and DynamoDB for managing data. Med-track allows patients to register, log in, book appointments, and submit diagnosis reports online. The system ensures real-time notifications, enhancing communication between doctors and patients regarding appointments and medical submissions. Additionally, AWS Identity and Access Management (IAM) is employed to ensure secure access control to AWS resources, allowing only authorized users to access sensitive data. This cloud-based solution improves accessibility and efficiency in healthcare services for all users..

Scenario 1: Efficient Appointment Booking System for Patients

In the MEDTRACK system, AWS EC2 provides a reliable infrastructure to manage multiple patients accessing the platform simultaneously. For example, a patient can log in, navigate to the appointment booking page, and easily submit a request for an appointment. Flask handles backend operations, efficiently retrieving and processing user data in real-time. The cloud-based architecture allows the platform to handle a high volume of appointment requests during peak periods, ensuring smooth operation without delays.

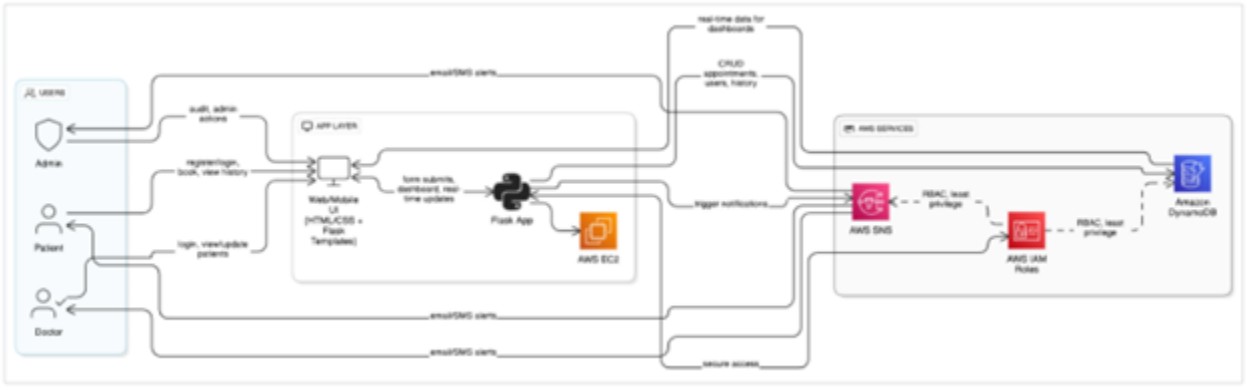
Scenario 2: Secure User Management with IAM

The MEDTRACK system provides doctors and patients with easy access to medical histories and relevant resources. For example, a doctor logs in to view a patient's medical history and upcoming appointments. They can quickly access, and update records as needed. Flask manages real-time data fetching from DynamoDB, while EC2 hosting ensures the platform performs seamlessly even when multiple users access it simultaneously, offering a smooth and uninterrupted user experience.

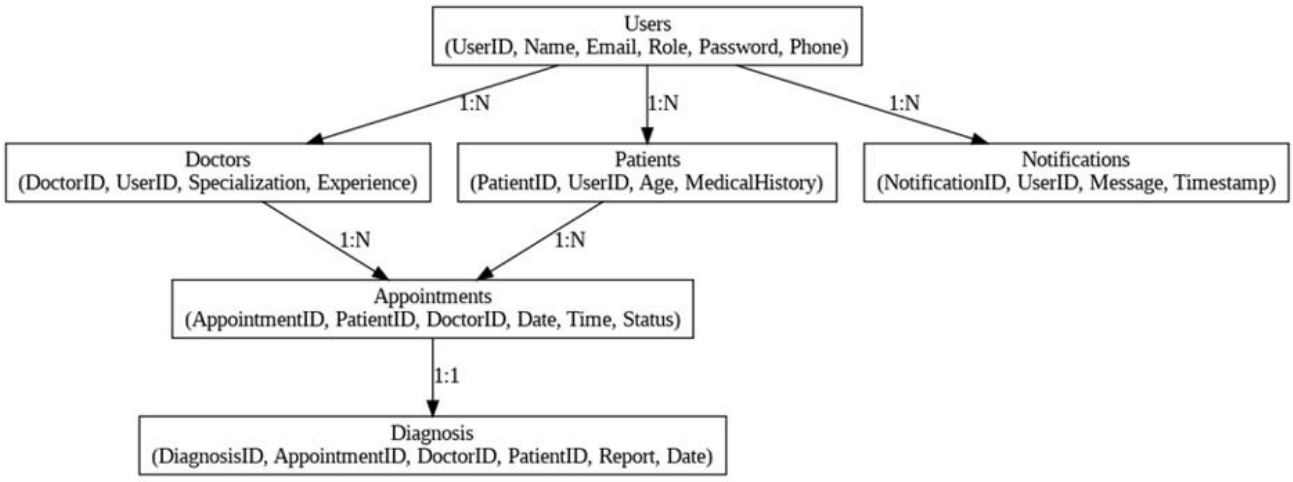
Scenario 3:Easy Access to Medical History and Resources

MEDTRACK features a dynamic user dashboard that displays all past and upcoming bookings for the logged-in user. For example, a user who has booked a flight and a hotel can view these bookings categorized by type, along with dates, price, and cancellation options. Flask fetches this data from AWS DynamoDB, which persistently stores all user bookings. The dashboard UI, powered by responsive HTML/CSS and Flask templates, ensures users can review or manage bookings anytime, from any device, with real-time updates and quick cancellation workflows.

AWS ARCHITECTURE



Entity Relationship (ER)Diagram:



Pre-requisites:

1. .**AWS Account Setup**: AWS Account Setup
2. **Understanding IAM**: IAM Overview
3. **Amazon EC2 Basics**: EC2 Tutorial
4. **DynamoDB Basics**: DynamoDB Introduction
5. **SNS Overview**: SNS Documentation
6. **Git Version Control**: Git Documentation

Project WorkFlow:

1. AWS Account Setup and Login

Activity 1.1: Set up an AWS account if not already done**.**

Activity 1.2: Log in to the AWS Management Console

2. DynamoDB Database Creation and Setup

Activity 2.1: Create a DynamoDB Table.

Activity 2.2: Configure Attributes for travel-Users and Bookings Tables.

3. SNS Notification Setup

Activity 3.1: Create an SNS Topic named BookingConfirmation for sending notifications.

Activity 3.2: Subscribe email to the topic to receive booking and cancellation alerts.

4. Backend Development and Application Setup

Activity 4.1:Develop the Backend Using Flask.

Activity 4.2: Integrate AWS Services Using boto3.

5. IAM Role Setup

Activity 5.1: Create IAM Role

Activity 5.2: Attach Policies

6. EC2 Instance Setup

Activity 6.1: Launch an EC2 instance to host the Flask application.

Activity 6.2: Configure security groups for HTTP, and SSH access.

7. Deployment on EC2

Activity 7.1:Upload Flask

Files **Activity 7.2**: Run the

Flask App

8. Testing and Deployment

Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.

Epic 1 : Web Application Development And Setup

- **Activity 1.1: Develop Backend Using Flask.**

The backend of the MEDTRACK system is developed using the Flask framework in Python. Flask is used to handle application routing, manage user requests, process appointment booking operations, and interact with AWS services such as DynamoDB and SNS. The backend also manages patient records, doctor information, and diagnosis submissions.

- **Activity 1.2:Create Frontend Pages.**

the frontend of the application is developed using HTML, CSS, and JavaScript to provide an interactive user interface. The system includes pages such as home page, login page, registration page, patient dashboard, doctor dashboard, appointment booking page, and diagnosis submission page. These pages allow users to interact with the system easily.

- **Activity 1.3:Implement User Authentication**

User authentication is implemented to allow patients and doctors to securely access the system. Users can register with their details and log in using their credentials. Passwords are securely stored and validated during login to ensure safe access to the healthcare management system.

Activity 1.4 – Configure Application Structure

The project follows a structured directory format. A templates folder is used to store HTML pages, while the static folder contains CSS and JavaScript files. The main backend logic is implemented in the app.py file. This structure ensures better organization and maintainability of the project.

Activity 1.5 – Test Web Application Locally

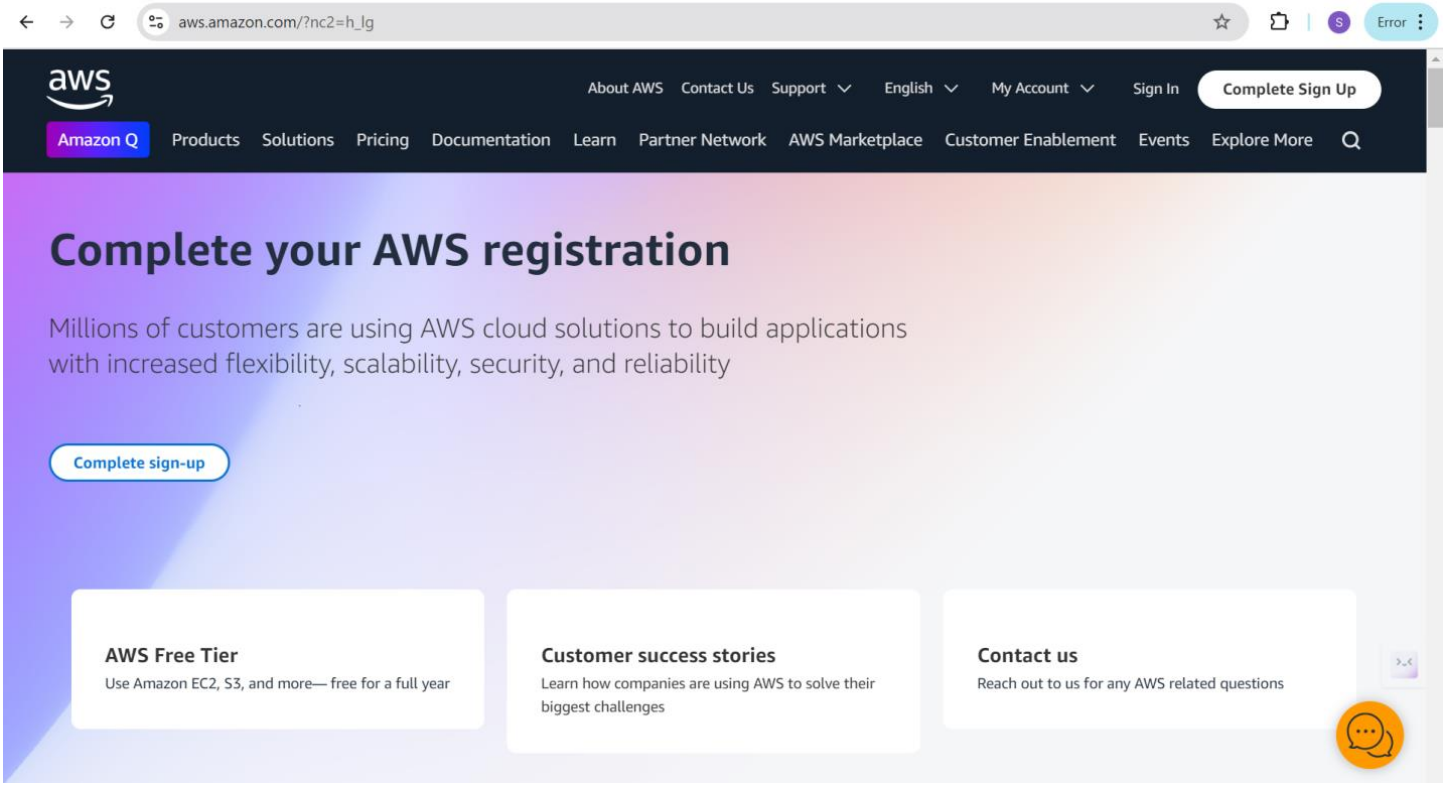
Before deploying the application to the cloud, the web application is tested locally. The Flask server is executed on the local machine to verify the functionality of user registration, login, appointment booking, and diagnosis submission. This ensures that the system works correctly before deployment.

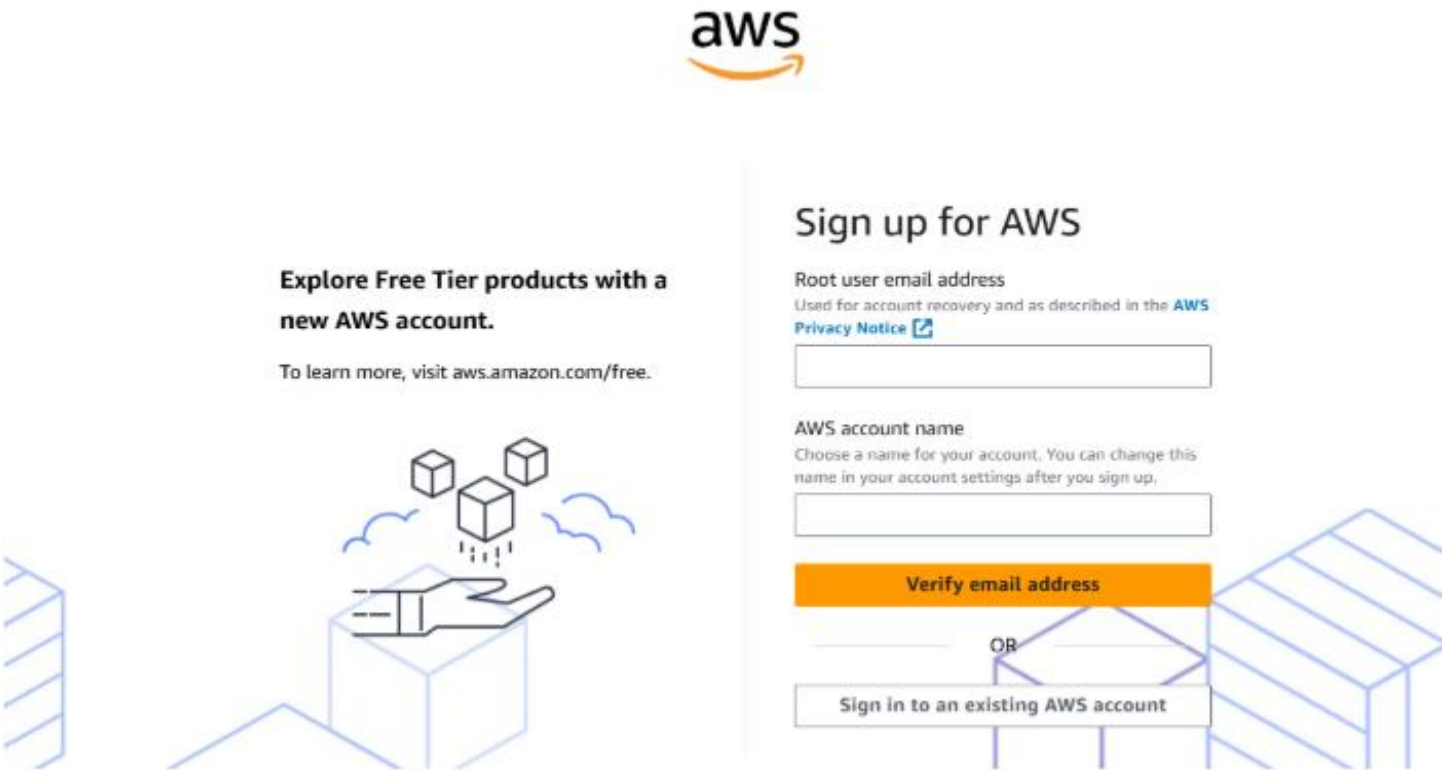
Epic 2 : AWS Account Setup

- **Activity 1.1: Set up an AWS account if not already done**.
    - Sign up for an AWS account and configure billing settings

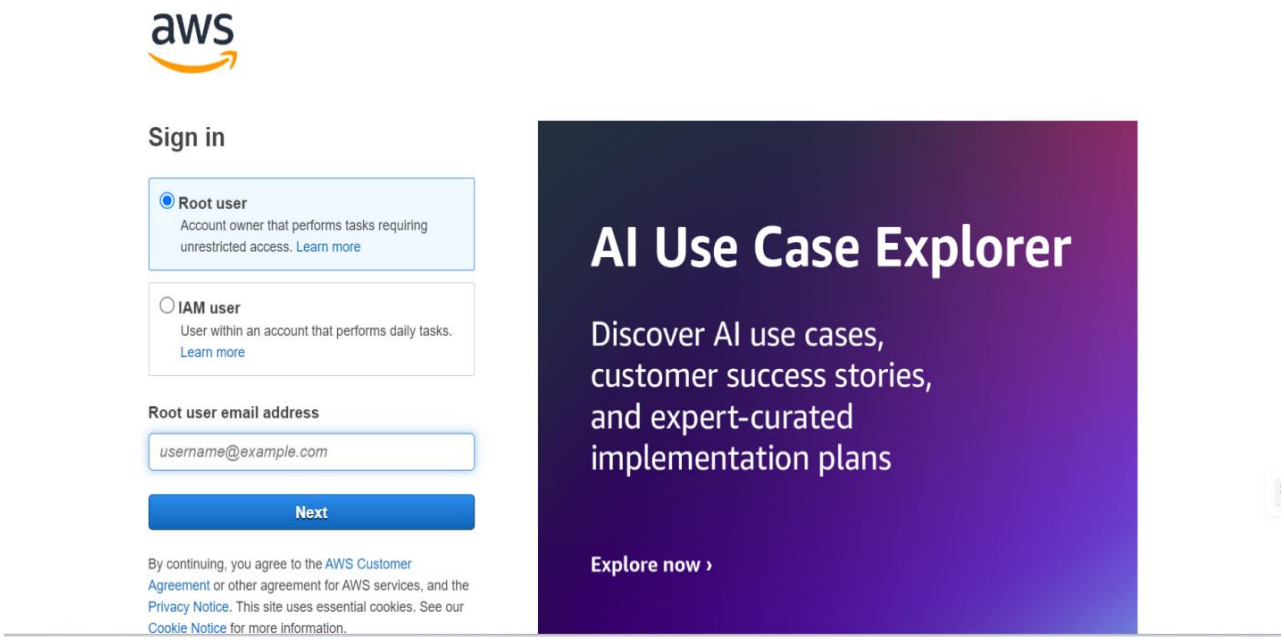
SMARTBRIDGE
Let's Bridge the Gap

Smart Internz





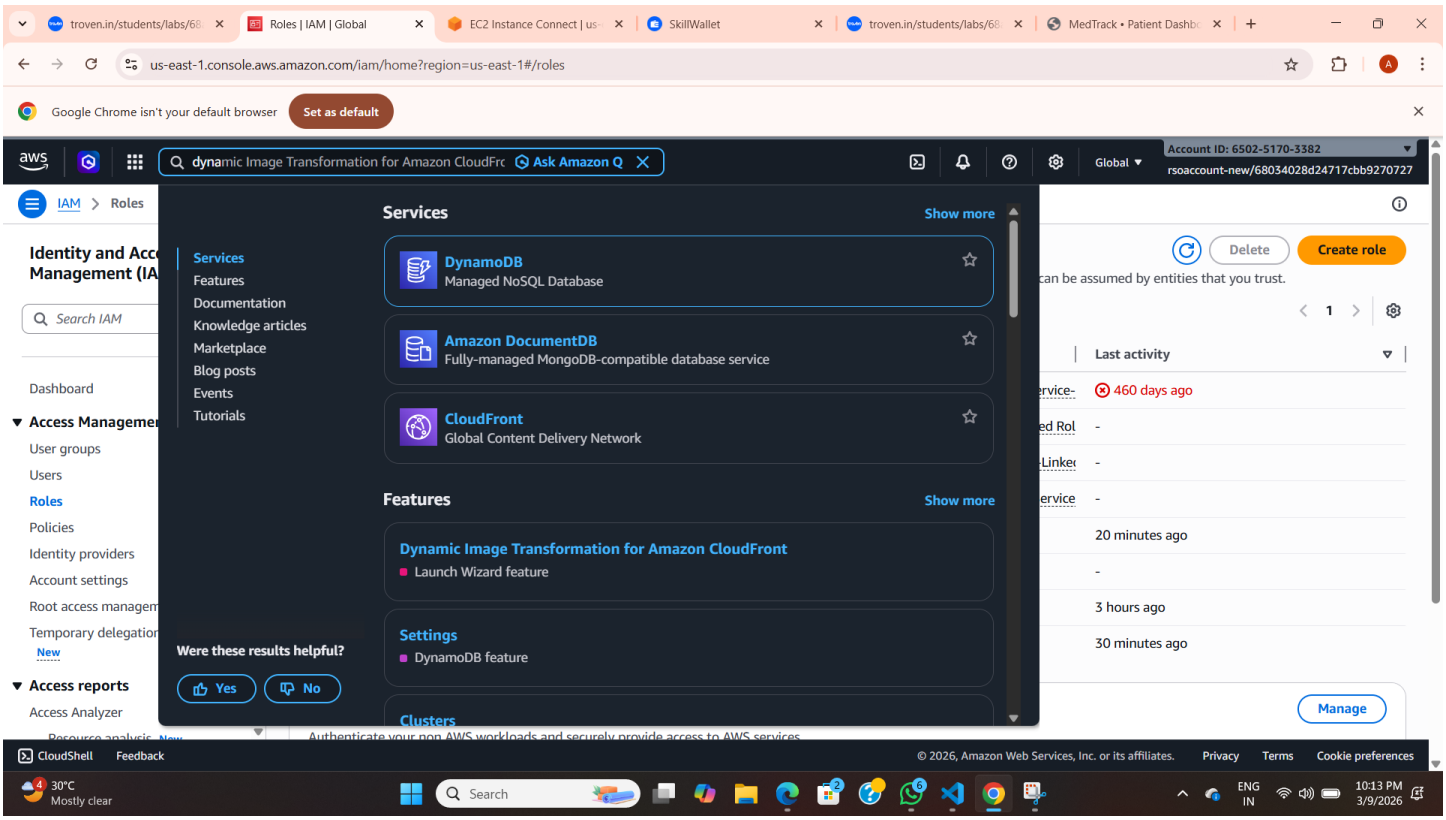
- **Activity 1.2: Log in to the AWS Management Console**
    - After setting up your account, log in to the AWS Management Console.



Epic 3: DynamoDB Database Creation and Setup

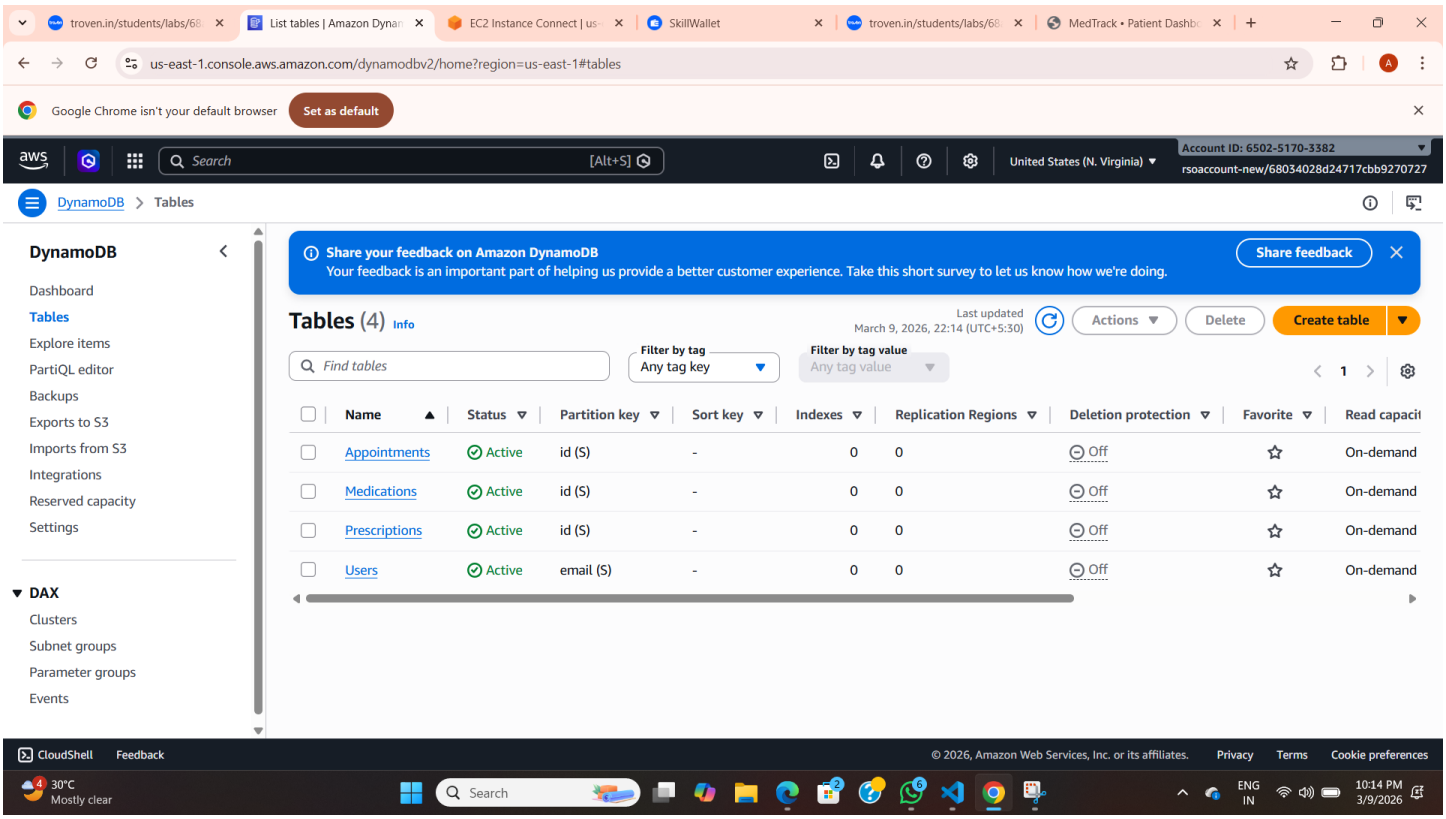
Activity 3.1:Navigate to the DynamoDB

To store user and appointment data for the MEDTRACK system, Amazon DynamoDB is used as the database service. In the AWS Management Console, the DynamoDB service is searched and selected to begin database creation and management.



Activity 3.2 – Create DynamoDB Tables

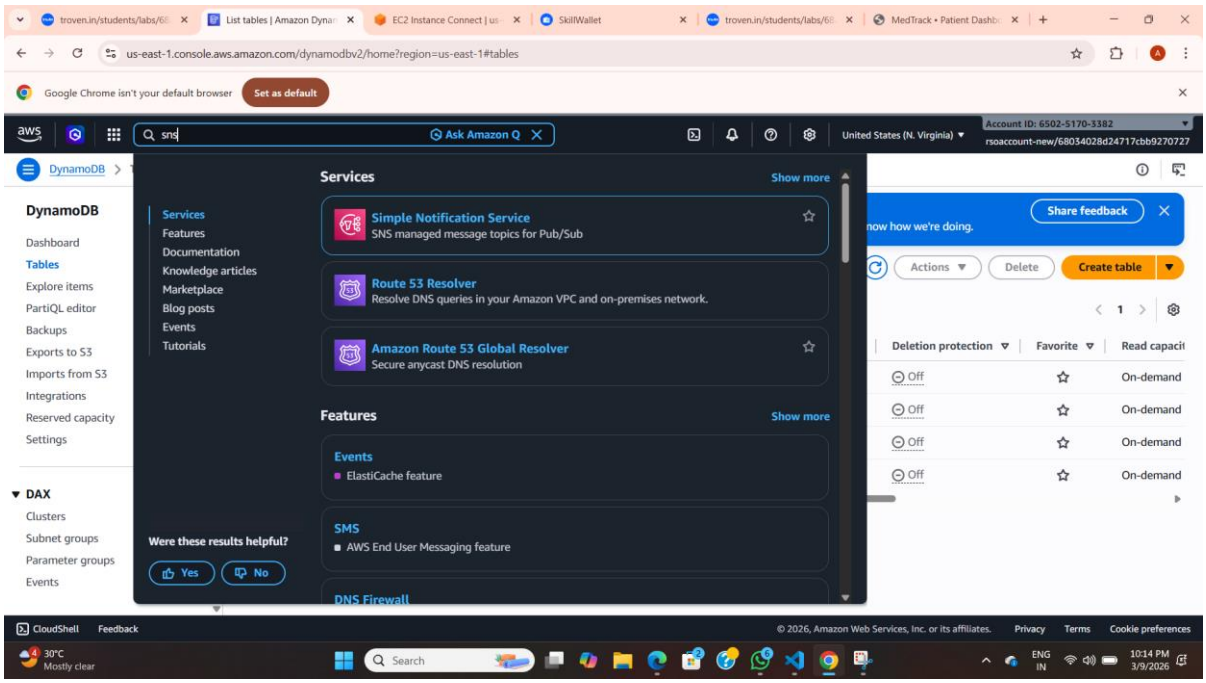
Two DynamoDB tables are created to store the system data. The Users table stores user information such as patient and doctor details. The Appointments table stores appointment booking details including appointment ID, doctor ID, patient ID, date, and status. These tables allow the system to efficiently manage healthcare records.


us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#tables
Google Chrome isn't your default browser
Set as default
Search
[Alt+S]
United States (N. Virginia)
Account ID: 6502-5170-3382
rsoaccount-new/68034028d24717cbb9270727
DynamoDB > Tables
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
Share your feedback on Amazon DynamoDB
Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.
Share feedback
Tables (4) Info
Last updated
March 9, 2026, 22:14 (UTC+5:30)
Actions
Delete
Create table
Find tables
Filter by tag
Any tag key
Filter by tag value
Any tag value
Name
Status
Partition key
Sort key
Indexes
Replication Regions
Deletion protection
Favorite
Read capaci
Appointments Active id (S) - 0 0 Off On-demand
Medications Active id (S) - 0 0 Off On-demand
Prescriptions Active id (S) - 0 0 Off On-demand
Users Active email (S) - 0 0 Off On-demand
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
30°C
Mostly clear
Search
ENG
IN
10:14 PM
3/9/2026

Epic 4: SNS Notification Setup

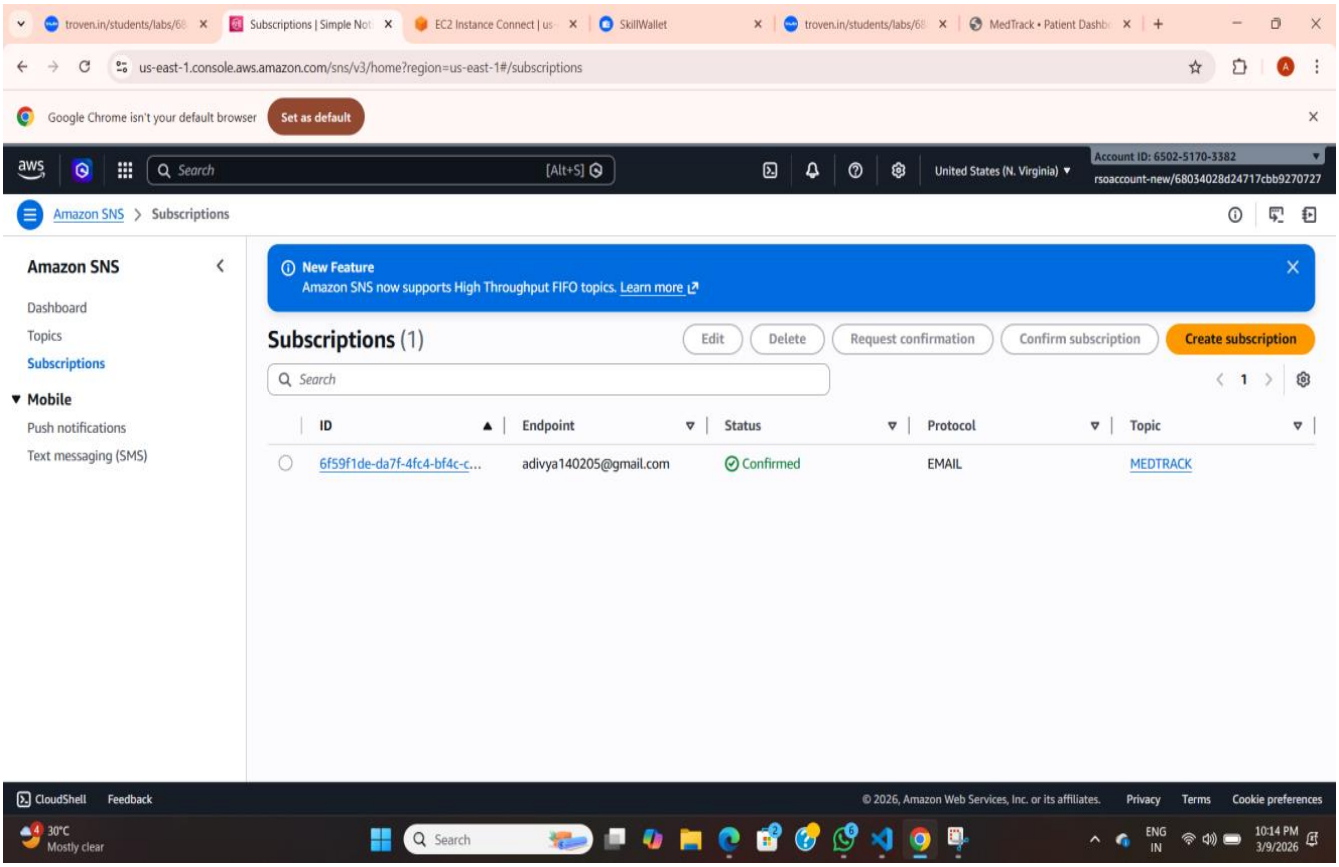
Activity 4.1 – Create SNS Topic for Sending Email Notifications

Amazon Simple Notification Service (SNS) is used in the MedTrack system to send notifications to users. An SNS topic is created in the AWS console to manage email notifications. This topic allows the system to send automated notifications when important actions occur such as appointment booking or diagnosis updates.



Activity 4.2 – Subscribe Users and Admin to SNS Topic

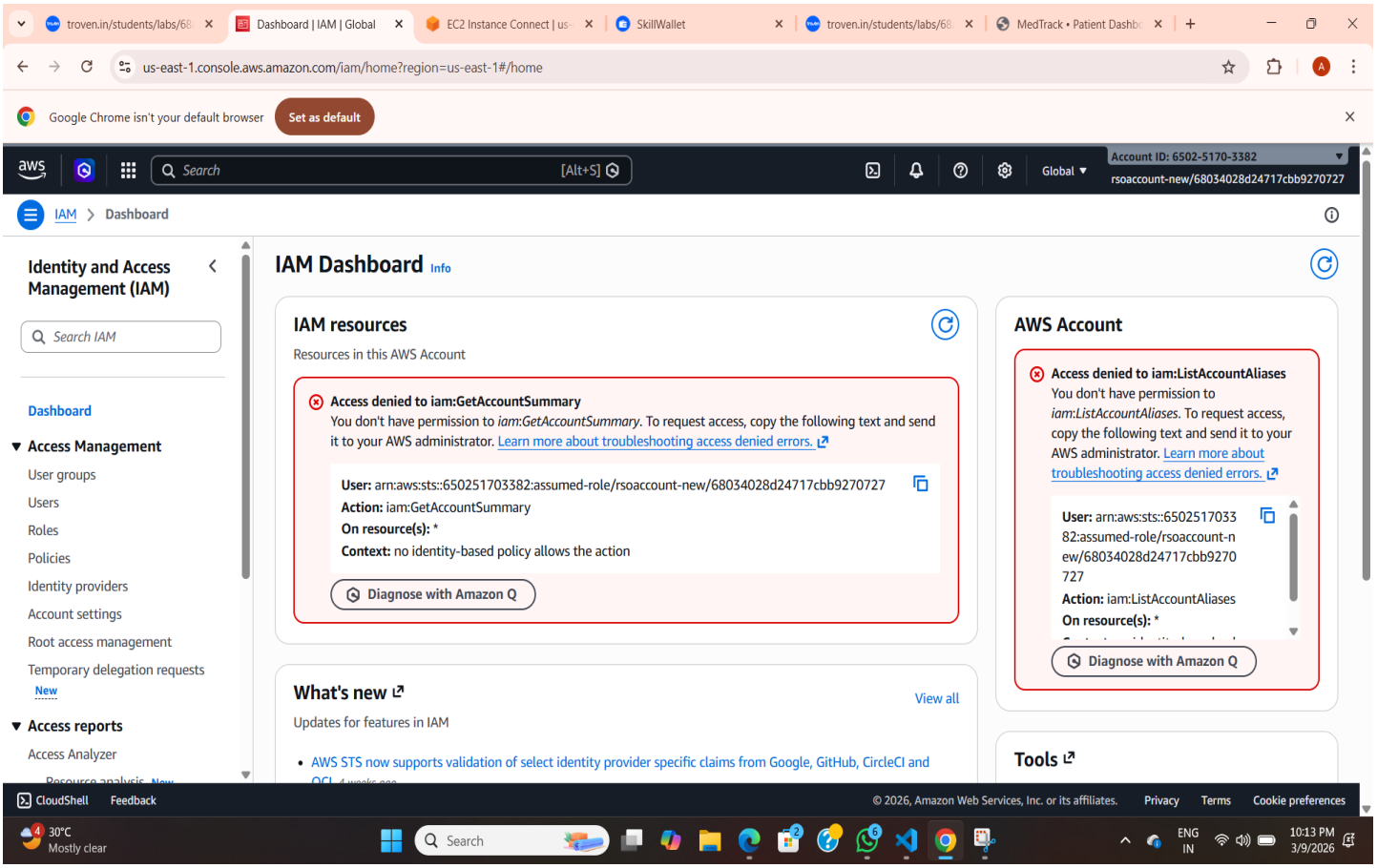
After creating the SNS topic, email addresses of users and administrators are subscribed to the topic. Once subscribed, they will receive notifications whenever the system publishes a message to the SNS topic. This helps keep both patients and administrators informed about system activities.


us-east-1.console.aws.amazon.com/sns/v3/home?region=us-east-1#/subscriptions
Google Chrome isn't your default browser
Set as default
Search
[Alt+S]
United States (N. Virginia)
Account ID: 6502-5170-3382
rsoaccount-new/68034028d24717cbb9270727
Amazon SNS > Subscriptions
Amazon SNS
Dashboard
Topics
Subscriptions
Mobile
Push notifications
Text messaging (SMS)
New Feature
Amazon SNS now supports High Throughput FIFO topics. Learn more
Subscriptions (1)
Edit
Delete
Request confirmation
Confirm subscription
Create subscription
Search
ID
Endpoint
Status
Protocol
Topic
6f59f1de-da7f-4fc4-bf4c-c...
adivya140205@gmail.com
Confirmed
EMAIL
MEDTRACK
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
30°C
Mostly clear
Search
ENG
IN
10:14 PM
3/9/2026

Epic 5: IAM Role Setup

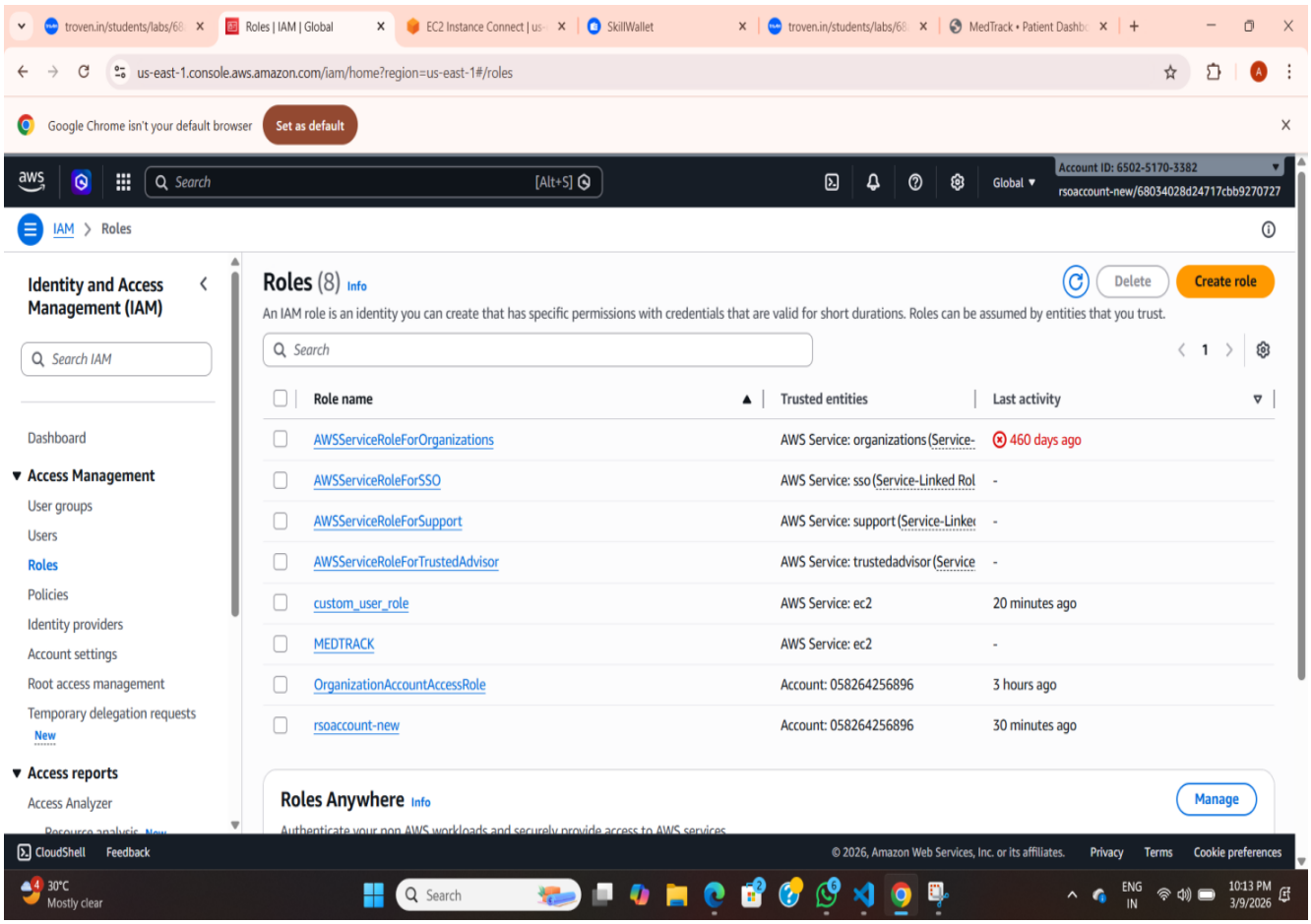
Activity 5.1 – Create IAM Role

AWS Identity and Access Management (IAM) is used to control access to AWS resources. An IAM role is created in the AWS Management Console to allow the EC2 instance to interact with other AWS services such as DynamoDB and SNS. This role ensures secure access and proper permissions for the application to function correctly.



Activity 5.2 – Attach Policies to IAM Role

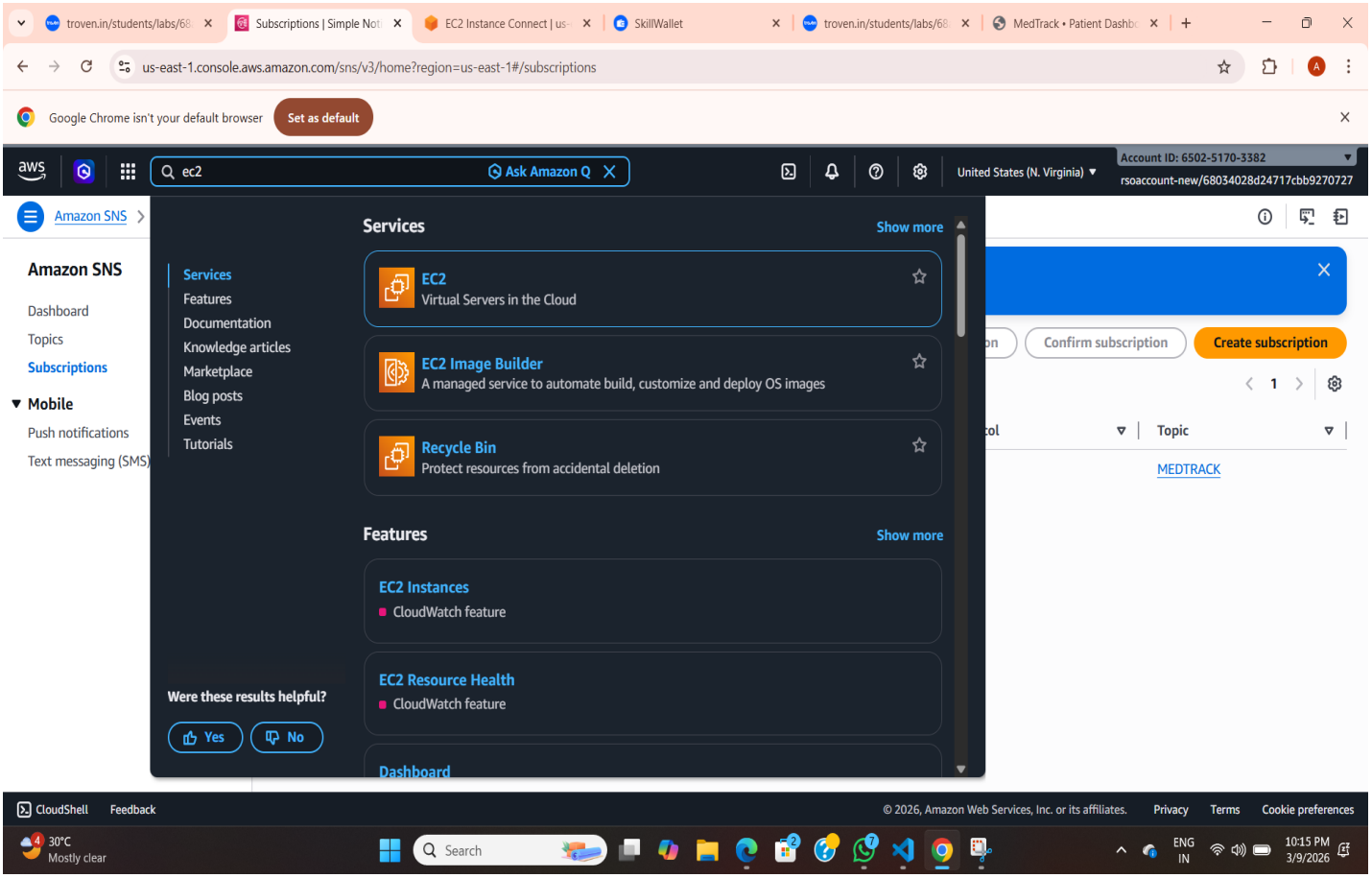
After creating the IAM role, required policies are attached to grant necessary permissions. Policies such as Amazon DynamoDB Full Access and Amazon SNS Full Access are added so that the application can read and write data in DynamoDB and send notifications using SNS.



Epic6: EC2 Instance Setup

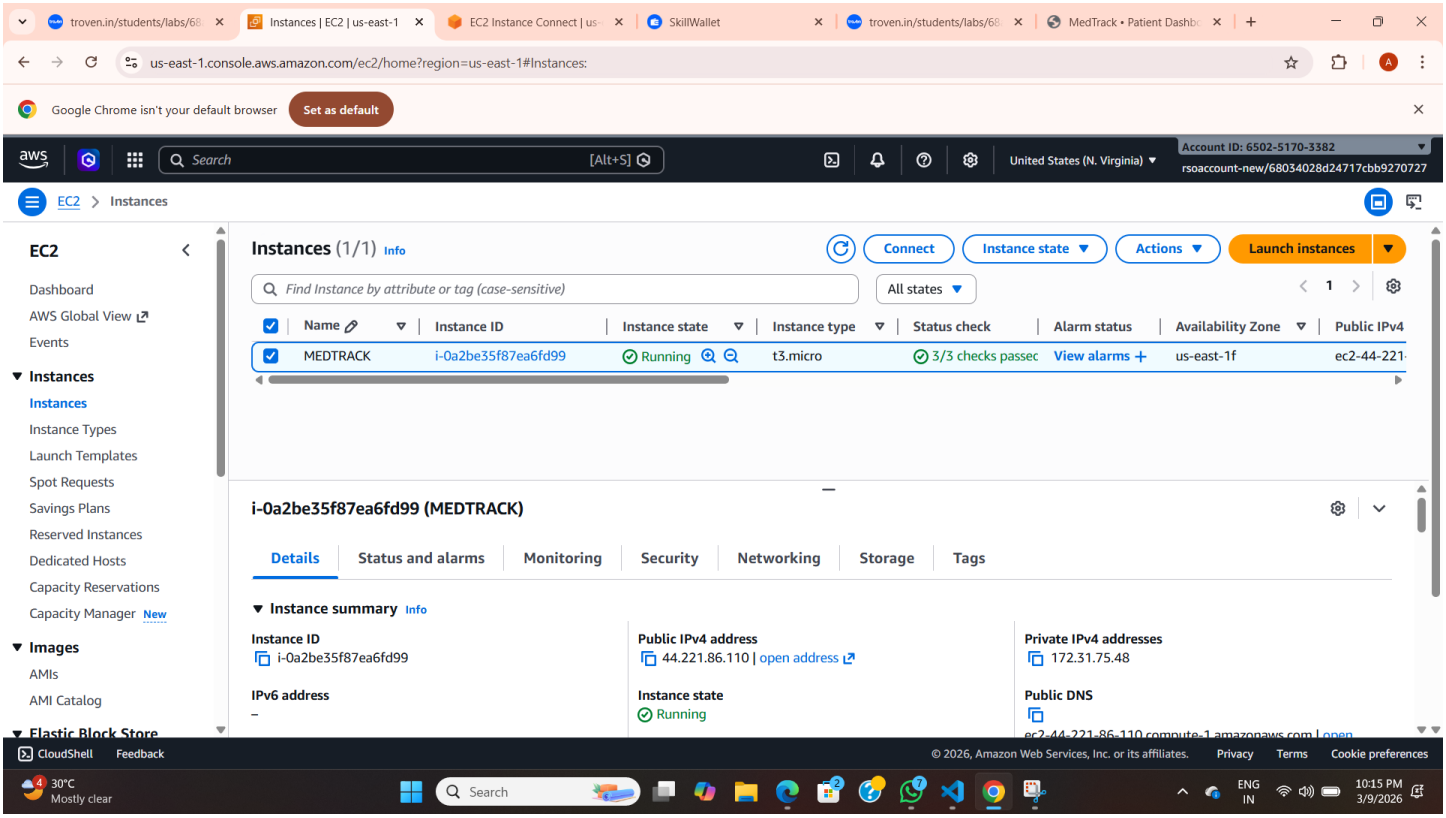
Activity 6.1 – Launch EC2 Instance

Amazon EC2 is used to host the MedTrack application on the cloud. In the AWS Management Console, the EC2 service is searched and selected. A new EC2 instance is launched by selecting an appropriate Amazon Machine Image (AMI) and instance type. This instance will run the Flask application and handle user requests.



Activity 6.2 – Configure Security Groups and Instance Settings

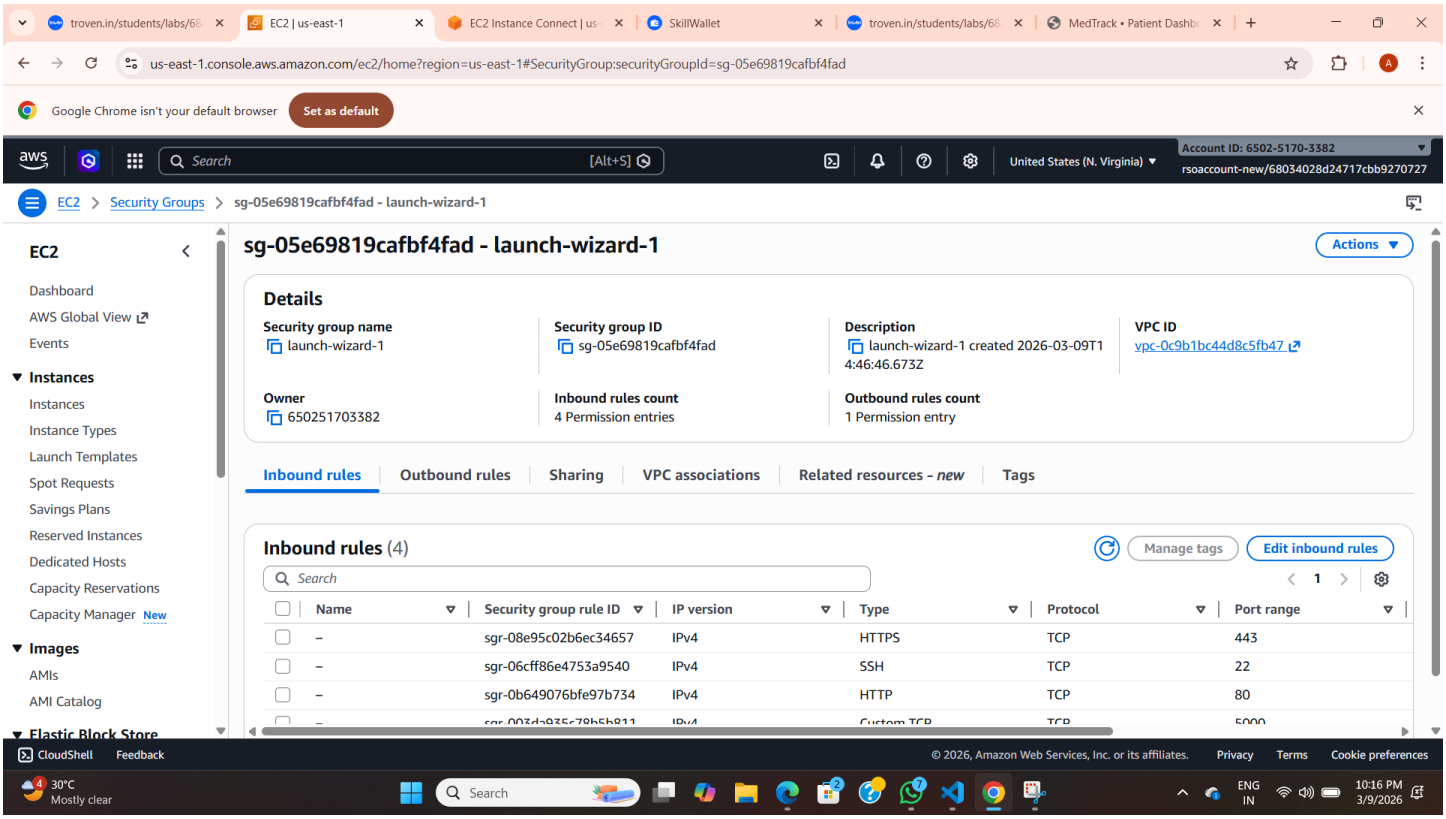
After launching the EC2 instance, security groups are configured to allow necessary network access. Ports such as SSH (22), HTTP (80), HTTPS (443), and custom application port (5000) are enabled so that the application can be accessed securely and remotely.



Epic 7: Security Group Configuration

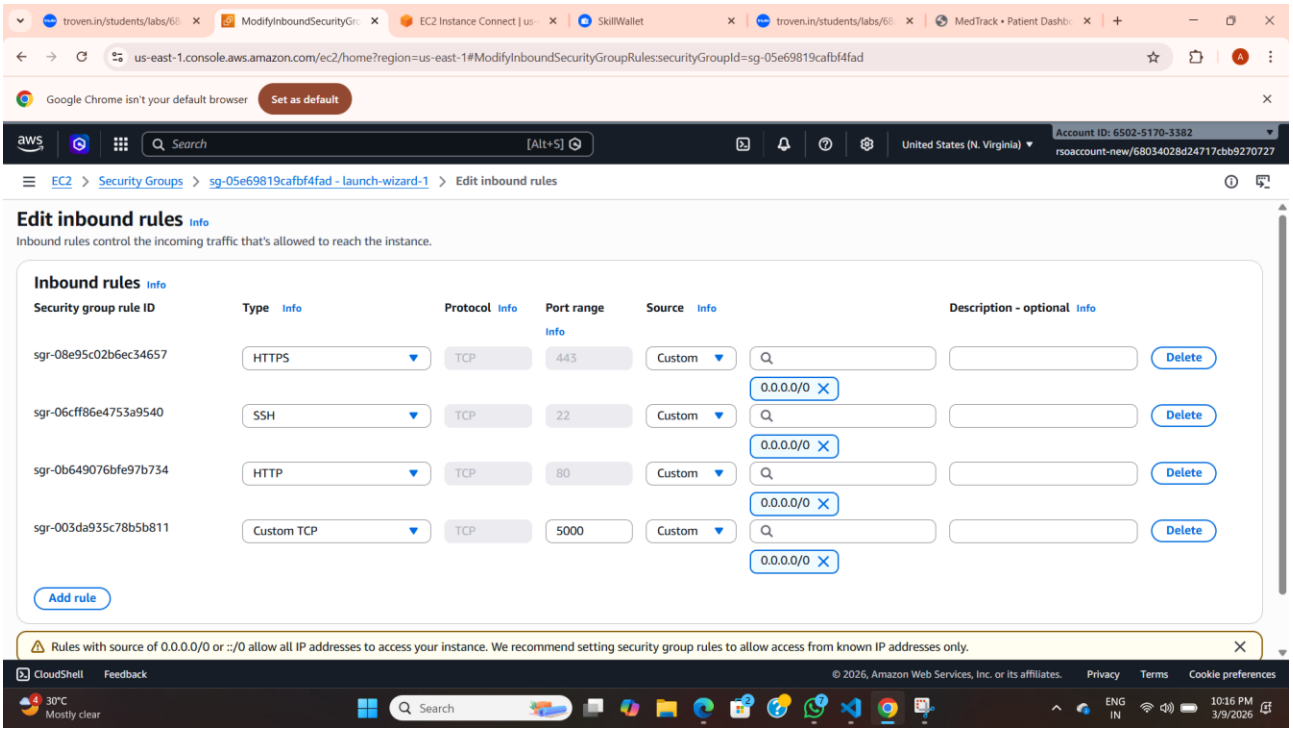
Activity 7.1 – View Security Group Details

After launching the EC2 instance, the security group attached to the instance is checked. Security groups act as virtual firewalls that control inbound and outbound traffic for the EC2 instance. The security group settings are reviewed to ensure the instance can communicate with required services.



Activity 7.2 – Configure Inbound Rules

Inbound rules are configured in the security group to allow necessary ports for accessing the application. Ports such as SSH (22) for remote access, HTTP (80) and HTTPS (443) for web access, and custom application port (5000) for the Flask application are enabled


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-05e69819cafbf4fad
Google Chrome isn't your default browser
Set as default
Search
[Alt+S]
United States (N. Virginia)
Account ID: 6502-5170-3382
rsoaccount-new/68034028d24717cbb9270727
EC2 > Security Groups > sg-05e69819cafbf4fad - launch-wizard-1 > Edit inbound rules
Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source Info
Description - optional Info
sgr-08e95c02b6ec34657
HTTPS
TCP
443
Custom
0.0.0.0/0
Delete
sgr-06cff86e4753a9540
SSH
TCP
22
Custom
0.0.0.0/0
Delete
sgr-0b649076bfe97b734
HTTP
TCP
80
Custom
0.0.0.0/0
Delete
sgr-003da935c78b5b811
Custom TCP
TCP
5000
Custom
0.0.0.0/0
Delete
Add rule
Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.
CloudShell
Feedback
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
30°C
Mostly clear
Search
ENG
IN
10:16 PM
3/9/2026

Epic 8: Deployment on EC2

Activity 7.1: Install Software on the EC2 Instance

 # Update and install Python + pip

sudo apt update

sudo apt install python3-pip -y

sudo apt install git -y

sudo apt install python3-venv -y

python3 -m venv venv

source venv/bin/activate

Activity 7.2:Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

Run: 'git clone https://github.com/adivya140205/MEDTRACK-AWS.git'

Note: change your-github-username and your-repository-name with your credentials here: 'https://github.com/AlekhyaPenubakula/Travel-Booking-website_Ec2_Dynamodb_SNS.git'

- This will download your project to the EC2 instance.

To navigate to the project directory, run the following command:

cd <your-folder>

Once inside the project directory, configure and run the Flask application by executing the following command with elevated privileges:

Run the Flask Application

export SNS_TOPIC_ARN=arn:aws:sns:ap-south-1:557690616836:BookingConfirmation

git pull origin main (Only if you made changes in git and want to reflect them in the EC2 terminal.)

 # Install requirements

pip install flask boto3

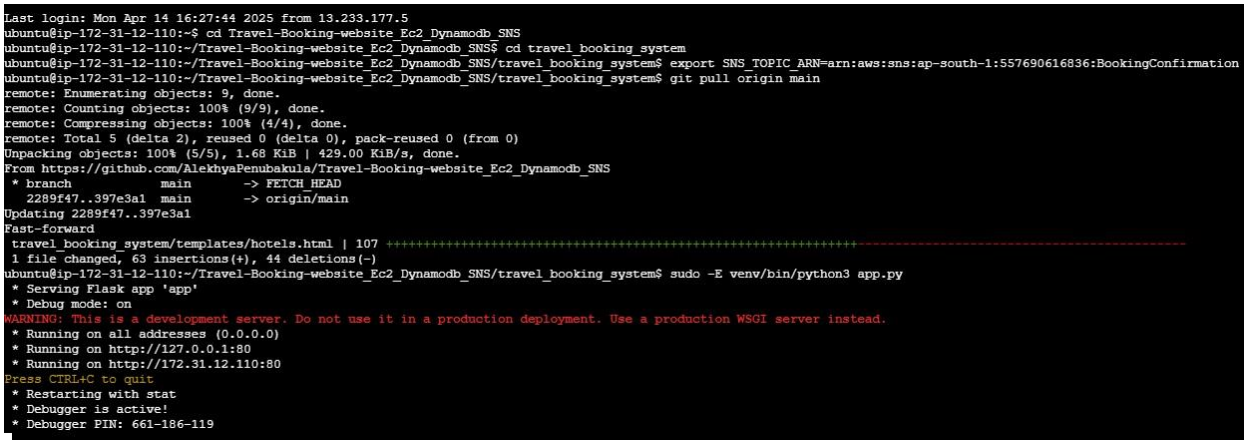
Launch the Flask app:

sudo -E venv/bin/python3 app.py

Verify the Flask app is running:

http://your-ec2-public-ip

- Run the Flask app on the EC2 instance



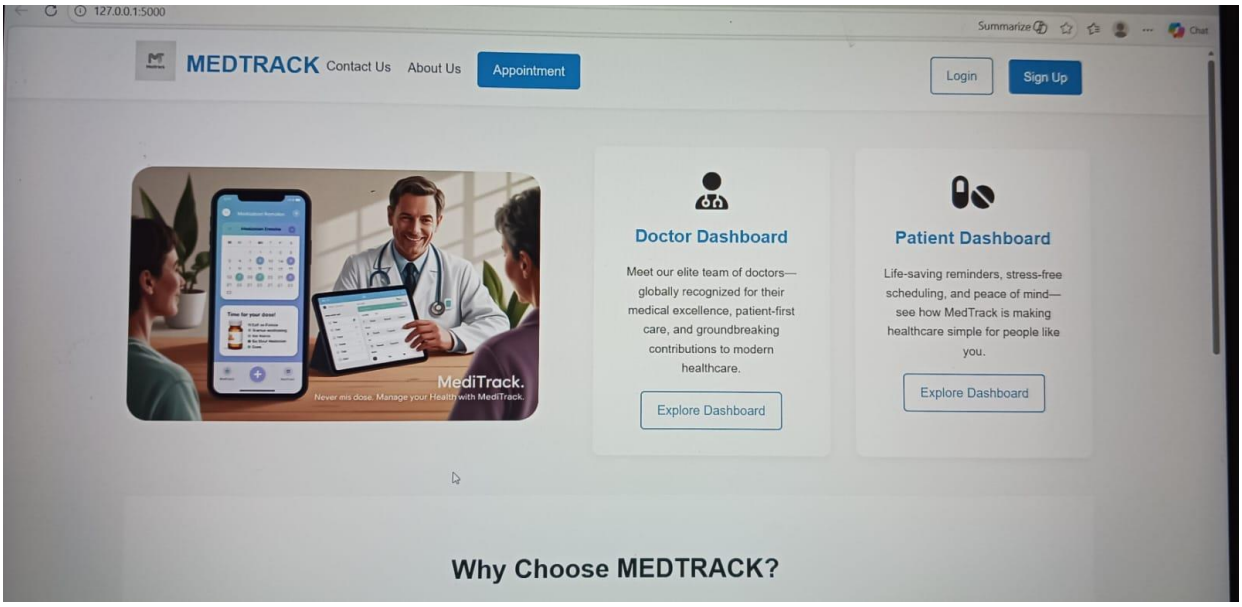
Access the website through:

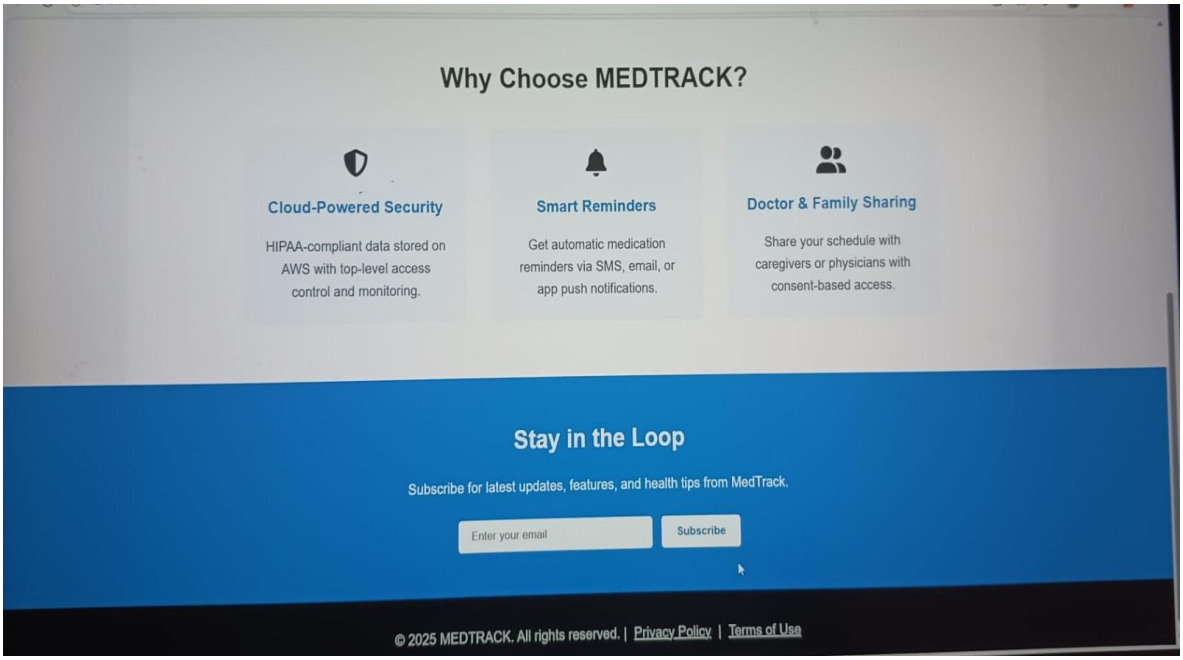
PublicIPS: http://44.221.86.110:5000

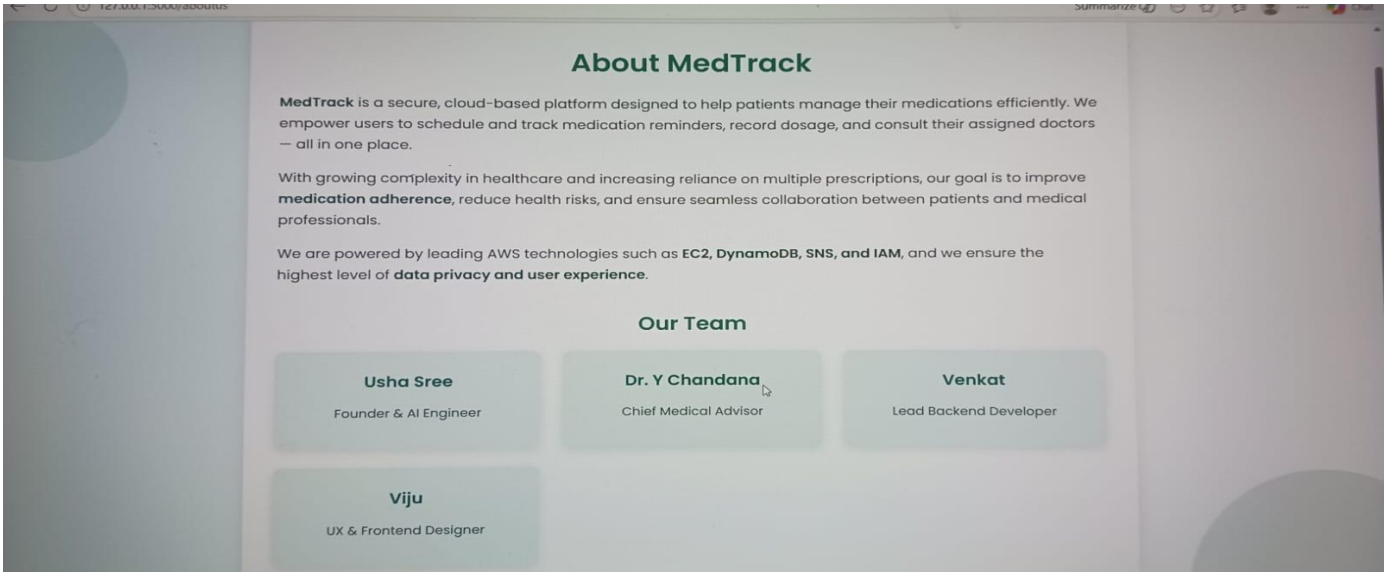
Milestone 8: Testing and Deployment

- **Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.**

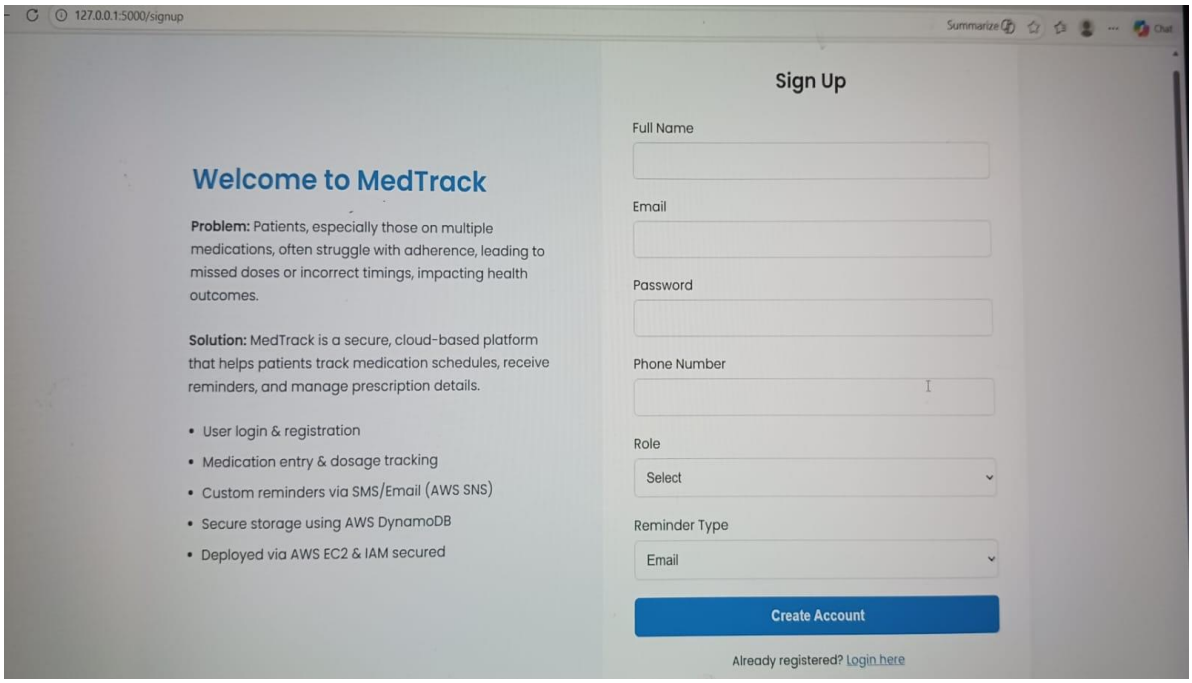
Home Page:



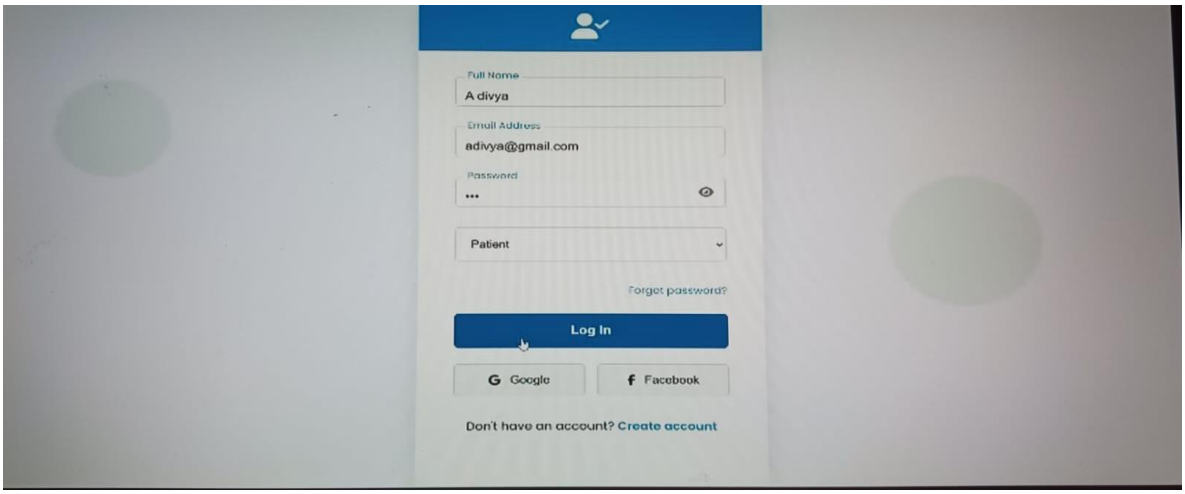




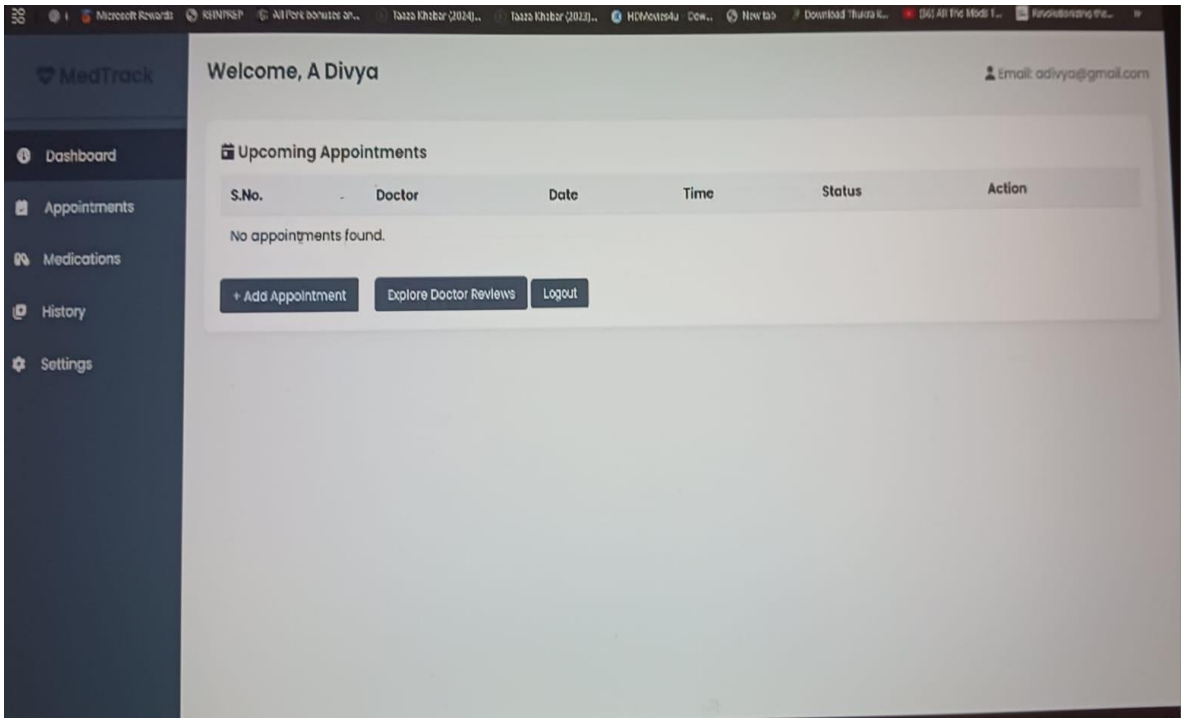
Register Page:

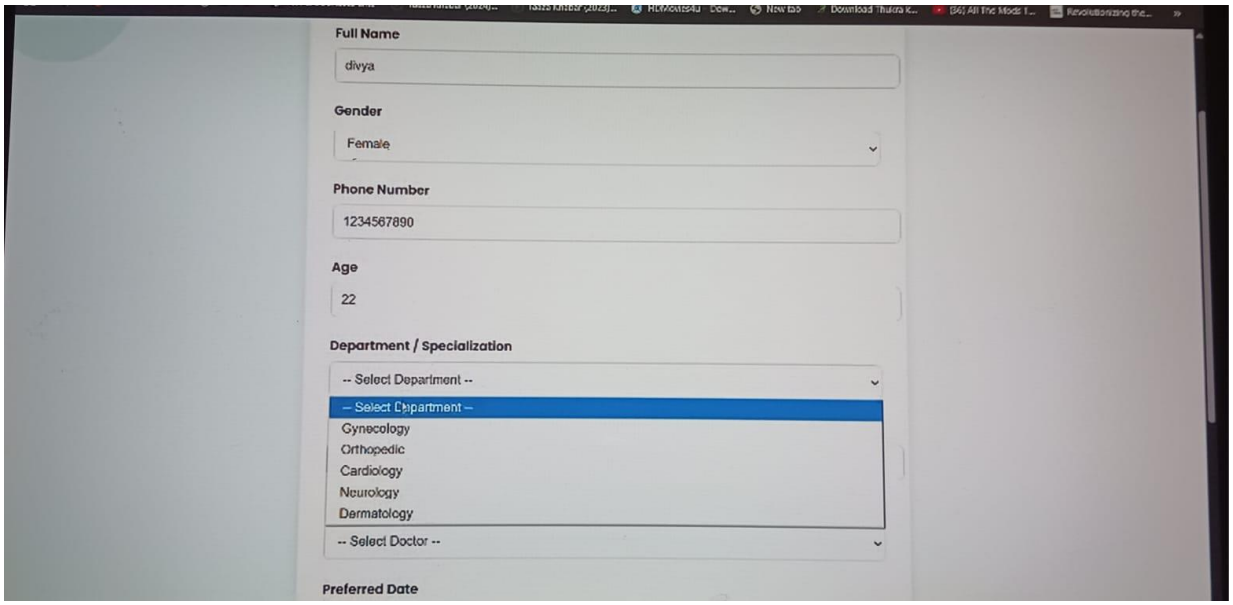


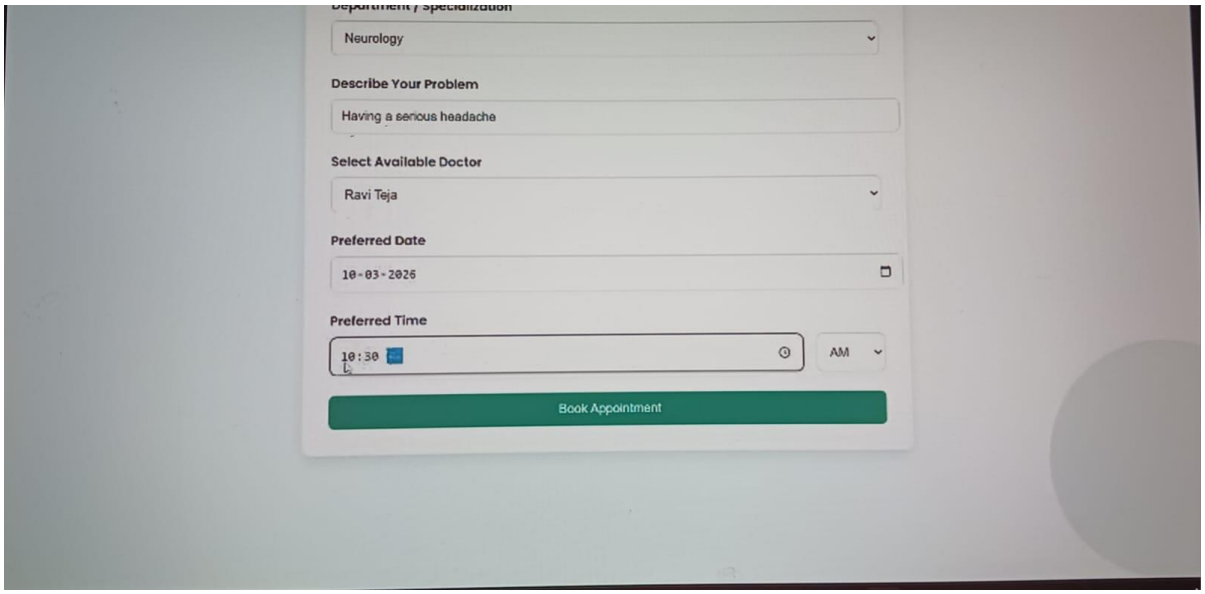
Register Page:



Dashboard page:

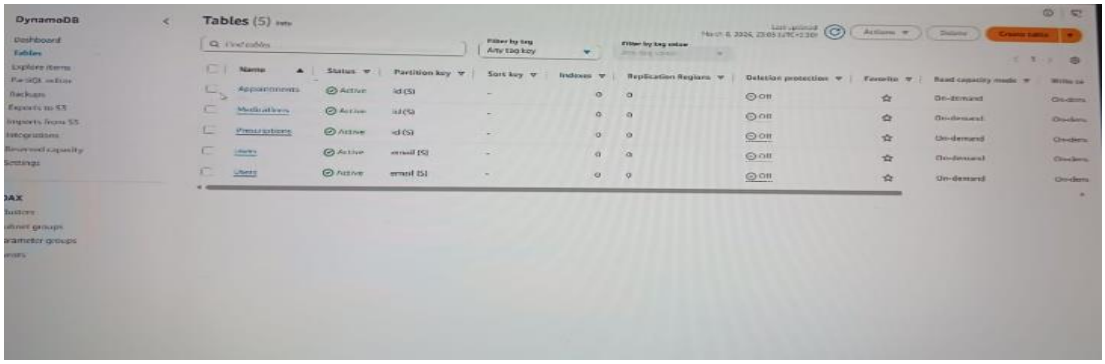



Full Name
divya
Gender
Female
Phone Number
1234567890
Age
22
Department / Specialization
-- Select Department --
-- Select Department --
Gynecology
Orthopedic
Cardiology
Neurology
Dermatology
-- Select Doctor --
Preferred Date


Neurology
Describe Your Problem
Having a serious headache
Select Available Doctor
Ravi Teja
Preferred Date
10-03-2026
Preferred Time
10:30
AM
Book Appointment

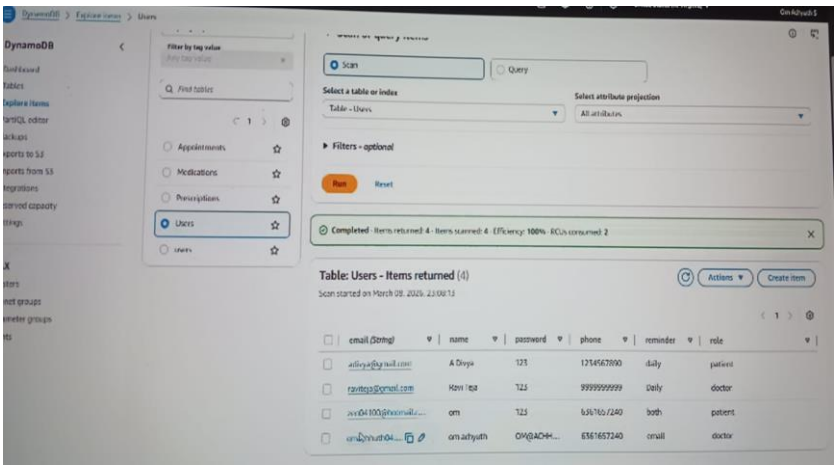
Dynamodb Database updations :

1. **Patient-users table :**



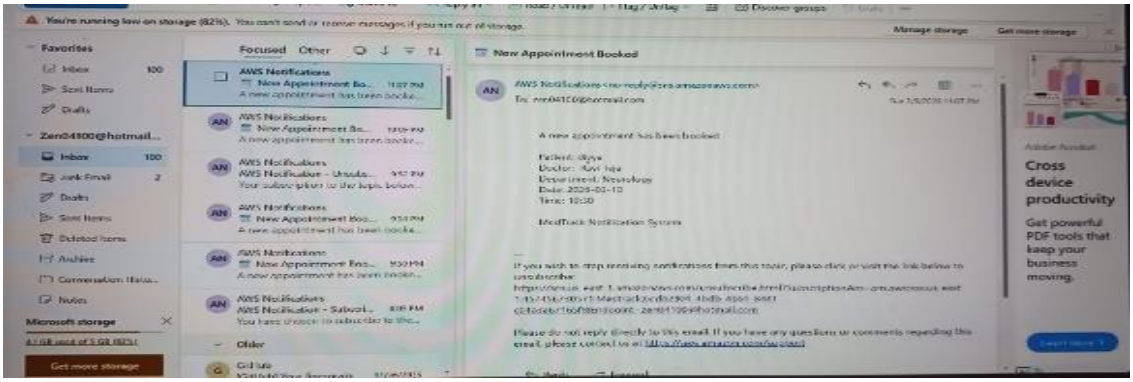
Description: Store and manage user credentials, login counts, and names in the Users table on DynamoDB for authentication and tracking purposes.

2. **bookings table :**



Description: Use the Bookings table in DynamoDB to store user travel bookings including date, destination, transport details, and payment information for each transaction.

3.Mail to the User:



Conclusion:

The MEDTRACK Healthcare Management System has been successfully designed and deployed using a cloud-based architecture on Amazon Web Services (AWS). The system integrates various AWS services such as EC2 for hosting the application, DynamoDB for efficient data storage, SNS for sending email notifications, and IAM for secure access management. By leveraging these cloud services, the platform ensures scalability, reliability, and secure handling of healthcare data.

The application provides an easy-to-use interface where patients can register, log in, and book appointments with doctors. Doctors can view appointment requests, access patient information, and update diagnosis reports. The system improves communication between patients and healthcare providers by providing a centralized platform for managing appointments and medical records.

Using Flask as the backend framework allows the application to efficiently process user requests and manage interactions between the web interface and AWS cloud services. DynamoDB enables fast and reliable storage of user and appointment data, while SNS ensures timely notifications to keep users informed about important system updates.

Overall, the MEDTRACK system demonstrates how cloud computing technologies can be effectively used to develop modern healthcare management solutions. The project highlights the benefits of using scalable cloud infrastructure to build secure, reliable, and user-friendly applications that enhance the efficiency of healthcare service.